Курс «Парадигмы и конструкции языков программирования»

Рубежный контроль №1

Вариант 18А: «Музыкальное произведение — Оркестр»

Задание

Созданы классы «Музыкальное произведение» (Work) и «Оркестр» (Orchestra), связанные отношением один-ко-многим, и класс «Репертуар оркестра» (WorkOrchestra) для связи многие-ко-многим. Количественный признак произведения — длительность исполнения в минутах. Запросы варианта А:

1. Один-ко-многим: список всех связанных произведений и оркестров, отсортированный по оркестрам.
2. Один-ко-многим: список оркестров с суммарной длительностью произведений, отсортированный по суммарной длительности.
3. Многие-ко-многим: список оркестров, в названии которых есть слово «оркестр», и произведений их репертуара.

Текст программы (rk1.py)

```
"""
Рубежный контроль №1 по курсу ПиКЯП.
Вариант 18А: предметная область «Музыкальное произведение — Оркестр»,
запросы варианта А.
"""
# используется для сортировки
from operator import itemgetter


class Work:
    """Музыкальное произведение"""

    def __init__(self, id, title, duration, orchestra_id):
        self.id = id
        self.title = title
        # Длительность исполнения в минутах (количественный признак)
        self.duration = duration
        # ID оркестра (для реализации связи один-ко-многим)
        self.orchestra_id = orchestra_id


class Orchestra:
    """Оркестр"""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class WorkOrchestra:
    """
    'Репертуар оркестра' для реализации
    связи многие-ко-многим
    """

    def __init__(self, orchestra_id, work_id):
        self.orchestra_id = orchestra_id
        self.work_id = work_id
```

```
# Оркестры
orchestras = [
    Orchestra(1, 'Симфонический оркестр филармонии'),
    Orchestra(2, 'Камерный оркестр консерватории'),
    Orchestra(3, 'Академическая капелла'),
    # Связаны с произведениями только через репертуар (многие-ко-многим)
    Orchestra(4, 'Ансамбль старинной музыки'),
    Orchestra(5, 'Оркестр народных инструментов'),
]

# Музыкальные произведения
works = [
    Work(1, 'Симфония № 5 (Бетховен)', 31, 1),
    Work(2, 'Болеро (Равель)', 15, 1),
    Work(3, 'Времена года (Вивальди)', 40, 2),
    Work(4, 'Серенада для струнных (Чайковский)', 30, 2),
    Work(5, 'Всенощное бдение (Рахманинов)', 60, 3),
]

# Репертуар оркестров
works_orchestras = [
    WorkOrchestra(1, 1),
    WorkOrchestra(1, 2),
    WorkOrchestra(1, 4),
    WorkOrchestra(2, 3),
    WorkOrchestra(2, 4),
    WorkOrchestra(3, 5),
    WorkOrchestra(4, 3),
    WorkOrchestra(5, 2),
]


def get_one_to_many(orchestras, works):
    """
    Соединение данных один-ко-многим.
    Возвращает список кортежей (произведение, длительность, оркестр).
    """
    return [(w.title, w.duration, o.name)
            for o in orchestras
            for w in works
            if w.orchestra_id == o.id]


def get_many_to_many(orchestras, works, works_orchestras):
    """
    Соединение данных многие-ко-многим.
    Возвращает список кортежей (произведение, длительность, оркестр).
    """
    many_to_many_temp = [(o.name, wo.orchestra_id, wo.work_id)
                         for o in orchestras
                         for wo in works_orchestras
                         if o.id == wo.orchestra_id]
    return [(w.title, w.duration, orchestra_name)
            for orchestra_name, orchestra_id, work_id in many_to_many_temp
            for w in works
            if w.id == work_id]


def task_a1(one_to_many):
    """
    Список всех связанных произведений и оркестров,
    отсортированный по оркестрам (порядок произведений произвольный).
    """
    return sorted(one_to_many, key=itemgetter(2))


def task_a2(orchestras, one_to_many):
    """
    Список оркестров с суммарной длительностью произведений в каждом,
    отсортированный по суммарной длительности (по убыванию).
    """
    # Длительности произведений каждого оркестра
    durations = {o.name: [duration for _, duration, name in one_to_many if name == o.name]
```

```
                for o in orchestras}
    # Оркестры без произведений не выводятся
    return sorted(((name, sum(d)) for name, d in durations.items() if d),
                  key=itemgetter(1), reverse=True)


def task_a3(orchestras, many_to_many, word='оркестр'):
    """
    Оркестры, в названии которых есть слово word,
    и список произведений их репертуара.
    """
    return {o.name: [title for title, _, name in many_to_many if name == o.name]
            for o in orchestras
            if word in o.name.lower().split()}


def main():
    """Основная функция"""
    one_to_many = get_one_to_many(orchestras, works)
    many_to_many = get_many_to_many(orchestras, works, works_orchestras)

    print('Задание A1')
    for row in task_a1(one_to_many):
        print(row)

    print('\nЗадание A2')
    for row in task_a2(orchestras, one_to_many):
        print(row)

    print('\nЗадание A3')
    for name, titles in task_a3(orchestras, many_to_many).items():
        print('{}:'.format(name))
        for title in titles:
            print('    {}'.format(title))


if __name__ == '__main__':
    main()
```

Результаты выполнения

```
$ python3 rk1.py
Задание A1
('Всенощное бдение (Рахманинов)', 60, 'Академическая капелла')
('Времена года (Вивальди)', 40, 'Камерный оркестр консерватории')
('Серенада для струнных (Чайковский)', 30, 'Камерный оркестр консерватории')
('Симфония № 5 (Бетховен)', 31, 'Симфонический оркестр филармонии')
('Болеро (Равель)', 15, 'Симфонический оркестр филармонии')

Задание A2
('Камерный оркестр консерватории', 70)
('Академическая капелла', 60)
('Симфонический оркестр филармонии', 46)

Задание A3
Симфонический оркестр филармонии:
    Симфония № 5 (Бетховен)
    Болеро (Равель)
    Серенада для струнных (Чайковский)
Камерный оркестр консерватории:
    Времена года (Вивальди)
    Серенада для струнных (Чайковский)
Оркестр народных инструментов:
    Болеро (Равель)
```